

ASSIGNMENT-7.1

RUPALI

2303A52308

Batch-43

- To identify and correct syntax, logic, and runtime errors in Python programs using AI tools.
- To understand common programming bugs and AI-assisted debugging suggestions.
- To evaluate how AI explains, detects, and fixes different types of coding errors.
- To build confidence in using AI to perform structured debugging practices.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to detect and correct syntax, logic, and runtime errors.
- Interpret AI-suggested bug fixes and explanations.
- Apply systematic debugging strategies supported by AI- generated insights.
- Refactor buggy code using responsible and reliable programming patterns.

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

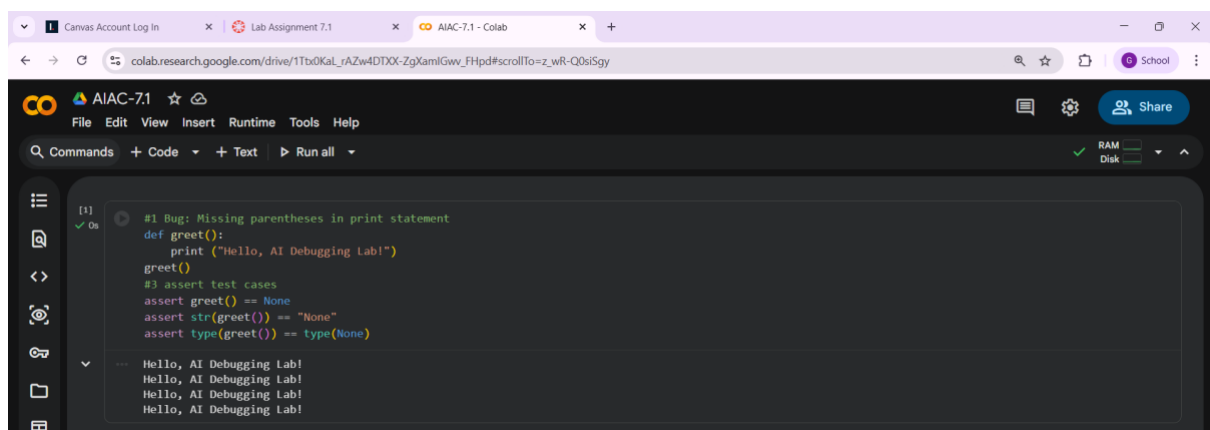
```
def greet():  
  
print "Hello, AI Debugging Lab!"  
  
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.



Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):  
  
if n = 10:  
  
    return "Ten"  
  
else:
```

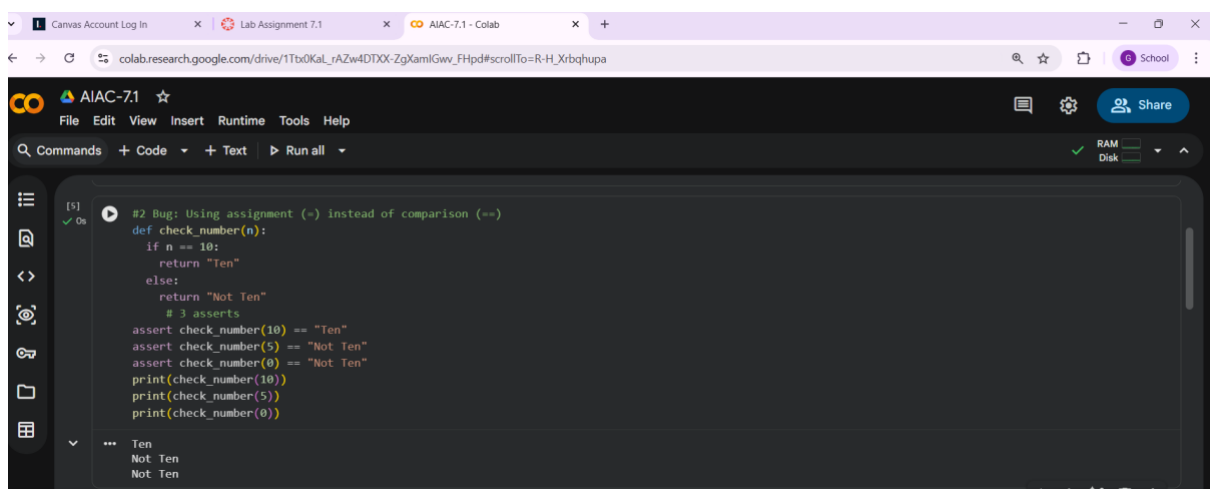
```
return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.



The screenshot shows a Google Colab notebook titled 'AIAC-7.1'. The code in the notebook is as follows:

```
[5] #2 Bug: Using assignment (=) instead of comparison (==)
def check_number(n):
    if n == 10:
        return "Ten"
    else:
        return "Not Ten"
    # 3 asserts
    assert check_number(10) == "Ten"
    assert check_number(5) == "Not Ten"
    assert check_number(0) == "Not Ten"
    print(check_number(10))
    print(check_number(5))
    print(check_number(0))

*** Ten
*** Not Ten
*** Not Ten
```

The output of the code execution is shown at the bottom of the cell, displaying 'Ten', 'Not Ten', and 'Not Ten' on separate lines.

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()

print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.

- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

The screenshot shows a Google Colab notebook titled "AIAC-7.1". The code defines a function `read_file(filename)` that uses a try-except block to handle `FileNotFoundError` and other exceptions, returning a user-friendly error message. The code then tests the function with three scenarios: a file that exists, a non-existent file, and an invalid path. The output shows the function successfully reading the existing file and returning the specified error messages for the missing file and invalid path.

```
[6] ✓ 0s
#3 Bug: Program crashes if file is missing
def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return "Error: File not found. Please check the filename or path."
    except Exception as e:
        return f"Error: {str(e)}"

# 3 Scenarios
print(read_file("sample.txt"))      # File exists
print(read_file("nonexistent.txt")) # File missing
print(read_file("C:/wrong/path/file.txt")) # Invalid path

*** Error: File not found. Please check the filename or path.
Error: File not found. Please check the filename or path.
Error: File not found. Please check the filename or path.
```

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:

def start(self):

return "Car started"

my_car = Car()

print(my_car.drive()) # drive() is not defined
```

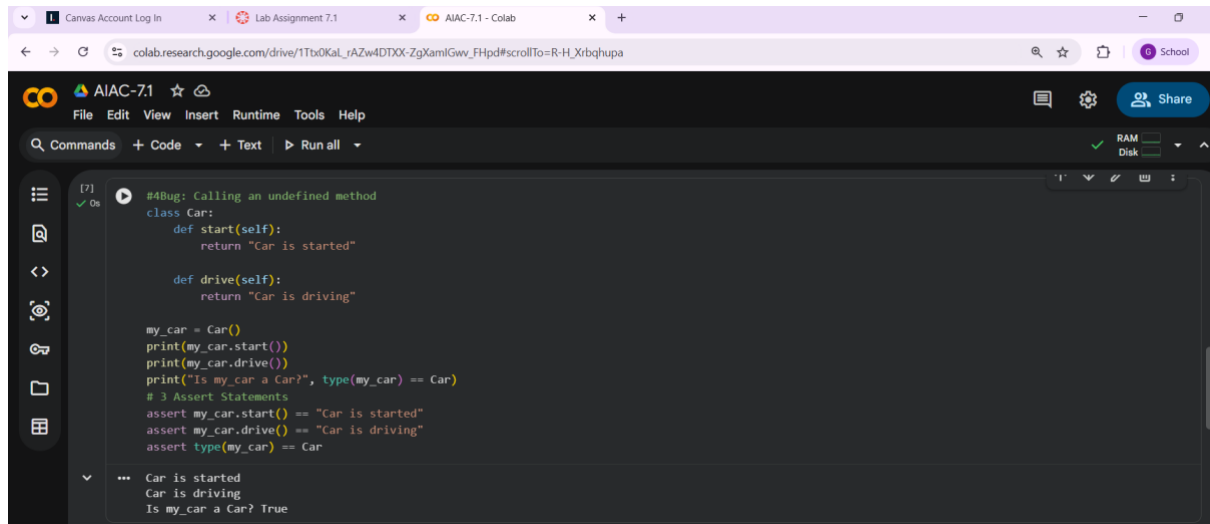
Requirements:

- Students must analyze whether to define the missing method or correct the method call.

- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.



```

#Bug: Calling an undefined method
class Car:
    def start(self):
        return "Car is started"

    def drive(self):
        return "Car is driving"

my_car = Car()
print(my_car.start())
print(my_car.drive())
print("Is my_car a Car?", type(my_car) == Car)
# 3 Assert Statements
assert my_car.start() == "Car is started"
assert my_car.drive() == "Car is driving"
assert type(my_car) == Car

... Car is started
... Car is driving
... Is my_car a Car? True
  
```

Task Description #5 (Type Error – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a Type Error. Use AI to resolve the bug.

Bug: Type Error due to mixing string and integer

```

def add_five(value):
    return value + 5

print(add_five("10"))
  
```

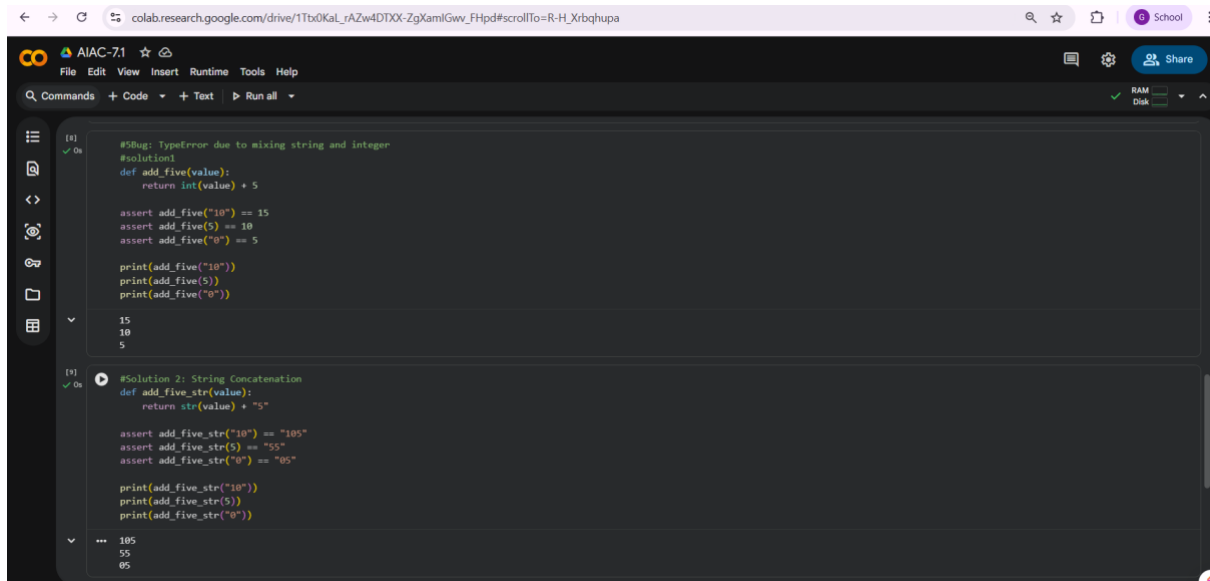
Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots



```
[8] ✓ On
#Bug: TypeError due to mixing string and integer
#solution1
def add_five(value):
    return int(value) + 5

assert add_five("10") == 15
assert add_five(5) == 10
assert add_five("0") == 5

print(add_five("10"))
print(add_five(5))
print(add_five("0"))

15
10
5

[9] ✓ On
#Solution 2: String Concatenation
def add_five_str(value):
    return str(value) + "5"

assert add_five_str("10") == "105"
assert add_five_str(5) == "55"
assert add_five_str("0") == "05"

print(add_five_str("10"))
print(add_five_str(5))
print(add_five_str("0"))

105
55
05
```